



NTU Water Resource Project • Final Report

台大水資源問題地圖

NTU Water Risk Map

校園即時回報、雨天導航、水災預警一站式平台

課程	水資源工程與管理（期末專題）
類型	Web Application — 互動式校園地圖平台
技術棧	Next.js 14 / TypeScript / React Leaflet / Recharts
資料源	OpenStreetMap、中央氣象署、台北自來水處 OpenData
部署	Vercel (hnd1 / 東京區域, 10 個 API Routes)
日期	2026 年 5 月



Make rainy-day campus walking less wet, one node at a time.

1. 摘要

本專題以「水資源問題」為核心，為台大師生開發一套**校園即時水情平台**——結合 眾包回報、即時氣象、校園路網最佳化與數據儀表板，把分散在校園各角落的積水、漏水、淹水資訊集中視覺化，並進一步轉化為「下一步該怎麼走、該怎麼避」的具體建議。

不同於傳統紙本問卷或單向公告，本平台採取**雙向數據迴圈**：使用者回報的問題會即時影響其他人的雨天路徑規劃；校方管理端可從儀表板與 CSV 匯出觀察高風險點，作為改善決策依據。全站共 6 個主要頁面、10 個 Node.js API Routes，整合 3 項外部資料源，路徑規劃部分以 Dijkstra (雙層 GeoJSON 路網，6,133 節點、7,248 條邊) 即時計算最低淋雨成本。

核心成果一覽

- **6 頁面**：地圖首頁、雨天路徑、水災預警、儀表板、個人成績單、管理後台
- **4 大問題類別**：淹水 / 積水 / 設施漏水 / 排水不良；3 級嚴重度與 4 種處理狀態
- **路徑演算法**：以 OSM tags 推論遮蔽率 (indoor/tunnel/covered/arcade)，雨大時可在僅多走 ~50 m 的條件下將遮蔽率提升 30%+
- **眾包飲水機地圖**：融合 OSM 社群 + 北水處官方 OpenData，26 台校內飲水機可即時報修、+1 減塑
- **減塑量化**：每 +1 換算為一個 600 mL 寶特瓶與 14 g CO₂e 節省量

2. 動機與問題定義

校園裡有水的問題嗎？有，而且很多。椰林大道兩側下大雨會嚴重積水、總圖前廣場常有低窪滯水、活大附近設施漏水、某些飲水機壞了沒人通報、學生不知道哪條路最少淋雨——這些都是真實但「知道的人很零散」的問題。

我們的觀察：資訊在校園裡是孤島。校方有官方申報系統但回報門檻高；學生在 Threads / Dcard 抱怨但訊息無法被追蹤；氣象署的雨量資料與校園座標毫無關聯。本專題希望以一個輕量、即時、地圖中心的 web app，把「**回報** → **視覺化** → **行為改變**」串成閉環。

三個底層問題與解法

1. **回報沒人理**：傳統表單填了之後不知道有沒有人看 ⇒ 用眾包確認機制 (*still_exists / resolved*) 讓社群相互驗證。
2. **雨天還是用 Google Maps**：但 GMaps 不知道椰林大道哪段有騎樓 ⇒ 用 OSM 抓的密集路網 + 雨勢加權 *Dijkstra* 給出真正避雨的走法。
3. **看數據的人不在地圖前**：校方需要的是趨勢與排行 ⇒ 用 14 天趨勢 + 風險分數排行 + 處理 SLA 的儀表板。

3. 系統架構

採用 Next.js 14 App Router 的 **Server Components** + **邊緣化 API** 架構：圖資與演算法重的工作（3 MB GeoJSON 載入、Dijkstra 計算、CWA API 代理）放在 server-side；地圖互動與 UI 在 client。整體分為五層：**Client 頁面** → **API Routes** → **演算法層** → **內部資料** → **外部公開資料源**。

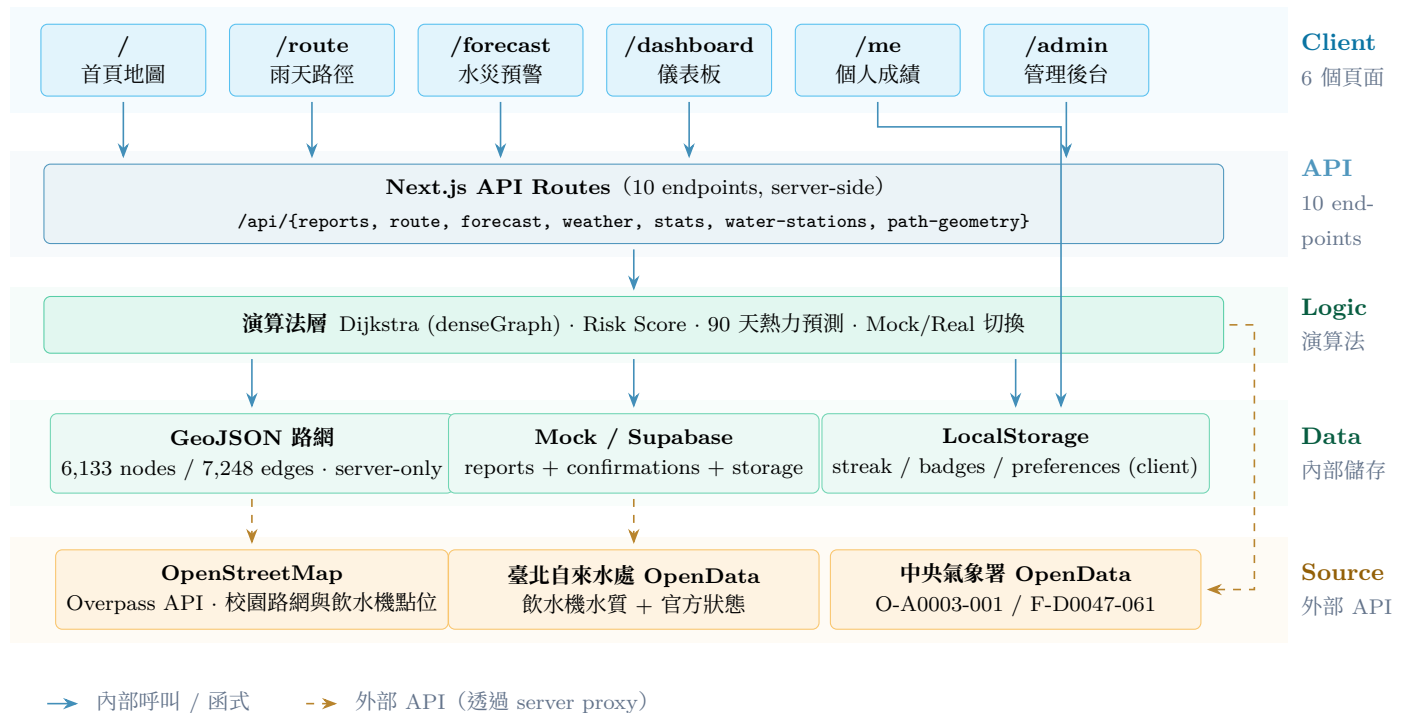


Figure 1 系統五層架構 (Swim-lane 視覺化)：Client 6 個頁面 → API Routes 10 endpoints → 演算法層 → 內部資料 (GeoJSON / Mock / LocalStorage) → 外部資料源 (OSM / 北水處 / CWA)。實線 = 內部呼叫，虛線 = 外部 API。/me 直連 LocalStorage 不經 server 以保護隱私。

3.1 技術選型理由

Next.js 14	App Router 提供 server-only 模組能力，3 MB 校園路網檔 (campus-paths.geojson) 只在 server 載入，client bundle 不受影響。
React Leaflet	開源、無 API key 限制，可疊加自訂 GeoJSON 與多型 marker；用 <code>dynamic({ssr:false})</code> 避開 SSR 的 window 問題。
Recharts	比 Chart.js 更貼合 React 心智模型，原生支援響應式容器，方便嵌入 dashboard 卡片。
Supabase (可選)	預設 <code>NEXT_PUBLIC_USE MOCK=true</code> ，零配置可 demo；接上即得 PostgreSQL + Storage + RLS。
Vercel hnd1	東京區域離台灣 RTT 最低，配合 <code>maxDuration:30s</code> 容許演算法 routes 載入大 GeoJSON。

3.2 資料流關鍵設計

- **Mock 與真實環境同介面**：`lib/supabase.ts` 透過環境變數切換，前端程式碼完全不變——demo 時零配置，正式上線時切 Supabase。
- **Server-side Heavy Compute**：Dijkstra 與 90 天熱力預測都在 `/api/route` 與 `/api/forecast` 計算，client 只收到 polyline 與機率格網，避免 3 MB 路網檔下載到瀏覽器。
- **個人資料純本機**：`/me` 的連續打卡、徽章、行動分數都用 `LocalStorage`，零追蹤、隱私第一。

4. 核心功能展示 — 5 個面向使用者的頁面，每節含實際畫面

4.1 地圖首頁 / —— 一張地圖看完校園水情

首頁採地圖中心（map-centric）設計：左側是互動式校園地圖，右側為四張資訊卡片（今日通勤、附近飲水機、高風險地點排行、最新回報）。使用者落地的 3 秒內可看到：(1) 校園有多少筆待處理問題、(2) 自己附近的飲水機狀態、(3) 該不該改走另一條路。

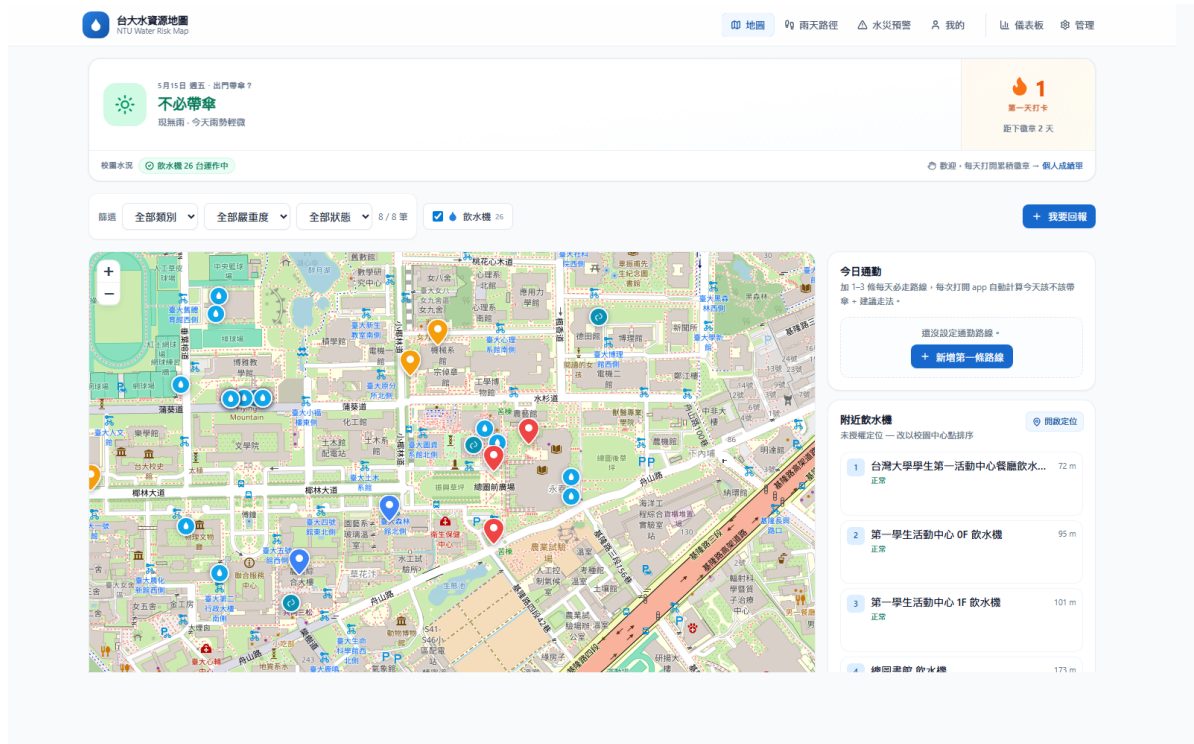


Figure 2 首頁實際畫面：地圖以台大校園為中心顯示水資源回報（紅／橙／藍 = 嚴重度）與飲水機點位（藍水滴），右側為高風險地點排行（依 risk_score 排序）。

首頁亮點功能

- 今日水情報（Today Briefing）：根據即時氣象在頁面頂端動態生成「不必擔心」「準備雨具」「強烈建議帶傘」等口語化提示。
- 雙層 markers 圖層切換：可獨立顯示／隱藏 26 台飲水機與校園水資源回報，避免視覺擁擠。
- 高風險地點排行（Risk Ranking）：依分組座標計算 $\text{risk_score} = \text{reports} \times 2 + \text{high} \times 3 + \text{recent7d} \times 2$ ，點擊即跳到地圖該位置。
- 回報眾包確認：每筆 report 支援「還在」「已解決」雙向投票，自動更新 upvote_count / resolved_count 。

4.2 雨天路徑規劃 /route —真實 OSM 路網 + 加權 Dijkstra

關鍵差異化：不是把 Leaflet polyline 畫一畫，而是用加權 *Dijkstra* 在真實 OSM 路網上做計算。地圖同時顯示防雨建議路徑（綠粗線）與最短路徑（灰虛線）作為對比，左側面板提供交通工具切換、雨勢 mock 滑桿、積水疊圖開關，並即時顯示遮蔽率提升、易積水比例下降、避開回報數。

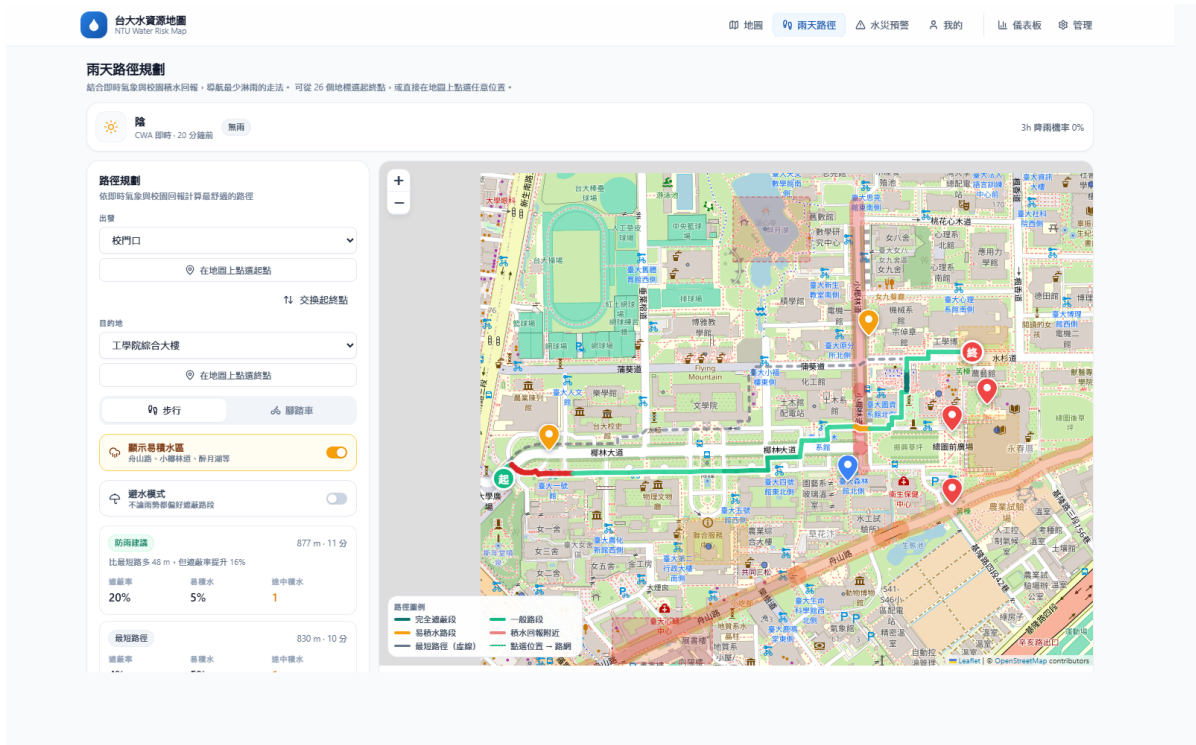


Figure 3 雨天路徑規劃畫面：綠粗線為防雨建議路徑，灰虛線為最短路徑對照。地圖支援「點任一處設為起終點」，server 會 snap 到最近的 OSM 路網節點。

邊權公式 (src/lib/denseGraph.ts)：

$$\text{cost} = d \times (1 + r \times (2.4(1-c) + 2.2\ell + 0.8f))$$

其中 d = 距離、 r = 雨勢係數 (CWA 即時雨量推算)、 c = 遮蔽率、 ℓ = 是否低窪、 f = 該段附近積水回報數。雨大時 r 變大，演算法會願意繞遠走有騎樓 / 室內走廊 (indoor=yes、arcade=yes、covered=yes) 的路徑。在中度雨勢 ($r = 0.6$) 下，防雨路徑相較最短路徑可達遮蔽率提升 $\sim 30\%$ ，但只多走 ~ 50 m。

4.3 水災預警地圖 /forecast —90 天回報密度 × 即時雨勢

水災預警地圖將整個校園分為 $\approx 20\text{ m}$ 的格網，融合歷史回報密度、地形低窪標記與即時雨勢，計算每格在未來 1 / 3 / 6 小時的積水機率。可切換 horizon 觀察短期 vs 中期風險，並在熱點位置上加上 marker 與彈出說明。

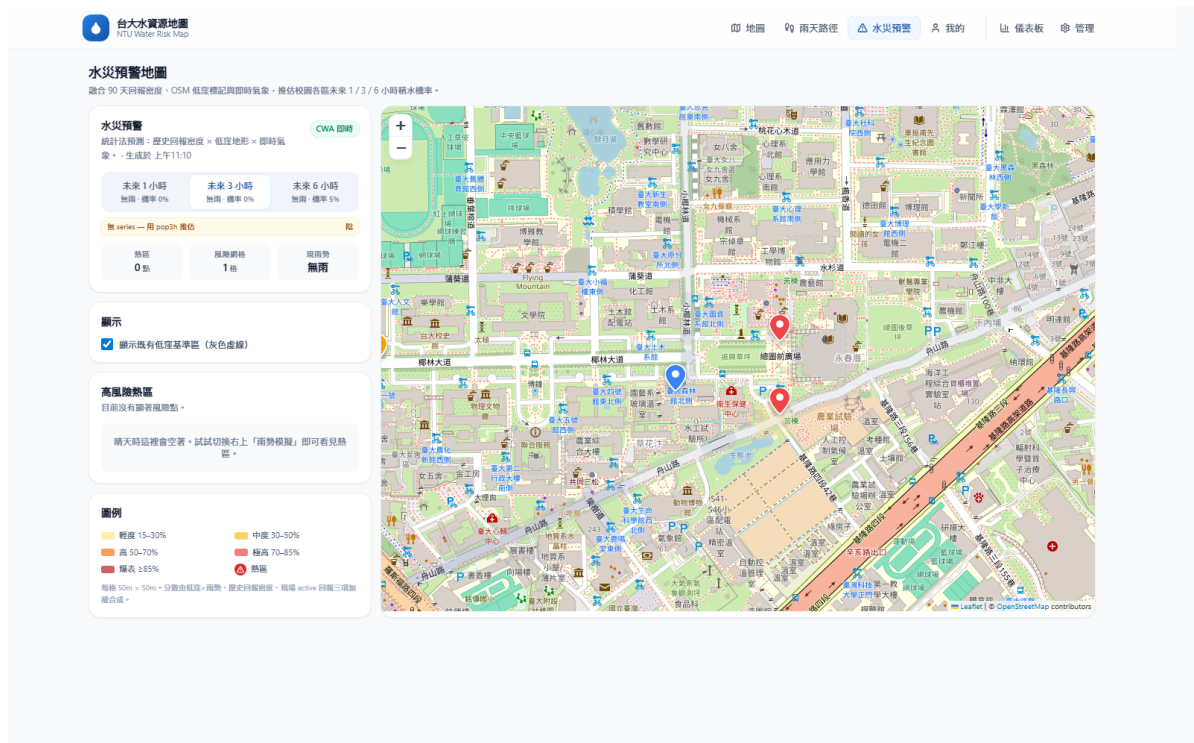


Figure 4 水災預警地圖：圖中所示為「不必擔心」狀態（雨勢小，仍標出 2 個基線熱點）；兩大時格網會自動加密與升溫。

預測模型不依賴黑盒 ML，而是可解釋的特徵組合：

- **歷史頻率**：將 90 天內的回報投影到 $\approx 20\text{ m}$ 格網，計算密度；同一格 5 筆遠重於 1 筆。
- **地形低窪**：由 floodAreas.ts 中手動標記的 5 個多邊形（小椰林道、醉月湖、總圖前廣場、舟山路低處、校門口）給予加成。
- **即時雨勢**：CWA 雨量乘上時間衰減（1h vs 6h 預測，越遠權重越低）。

4.4 校園水資源儀表板 /dashboard

儀表板不是 markers 的條形圖版，而是設計給「校方總務處決策者」看的：**解決率**、**平均處理天數**、**WoW 變化**（本週 vs 上週）、**飲水機 SLA**（故障件 / 待換濾心件）——都是直接對應 KPI 的指標。資料完全由 /api/stats 即時聚合，零快取，方便 demo 時 console 改 mock 立即生效。

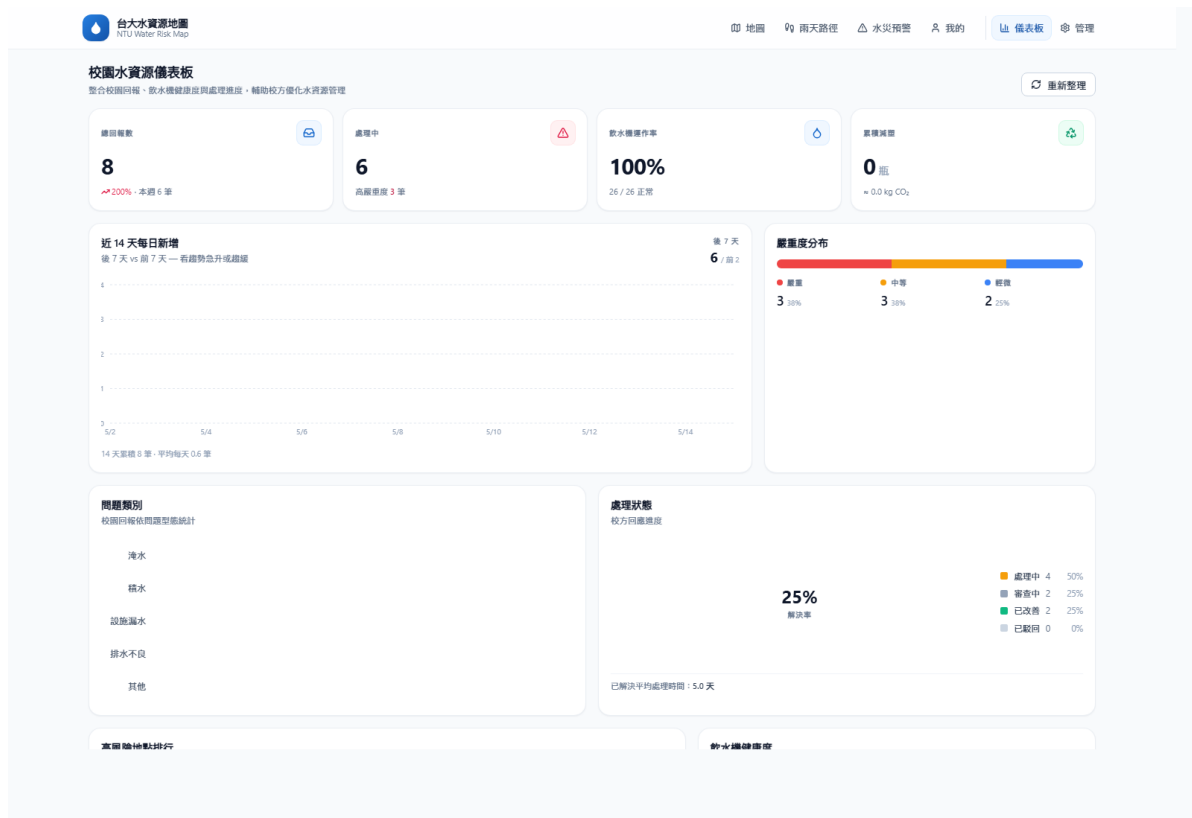


Figure 5 儀表板上半部：4 張 KPI 卡（總回報、處理中、飲水機健康率、寶特瓶減塑）、14 天每日新增、嚴重度分布、類別／處理狀態。下方還有高風險排行與飲水機健康度。

儀表板核心 KPI（給校方決策層看）

- **總回報數與 WoW 變化**：本週相比上週是急升還是趨緩？↑200% 直接觸發人力盤點。
- **處理中件數與 高嚴重度子集**：判斷積壓量；嚴重件需獨立處理流程。
- **飲水機運作率（健康件 / 總件）+ 累積減塑寶特瓶**：定期向永續長報告的 ESG 指標。
- **14 天日新增趨勢線**：早期辨識梅雨季 onset。

4.5 個人成績單 /me 一把回報遊戲化

設計動機：回報意願是專案存活的關鍵。我們把抽象的「做好事」轉化為立即可見的回饋——連續打卡天數（Streak）、行動分數（4 維加權）、徽章解鎖、社群分享文。14 枚徽章橫跨「連續打卡」「飲水機 / 減塑」「通勤路線」「校園回報」四個行為類別，從新手「初次入坑」到極限「一年水文皇帝」（連續打卡 365 天），涵蓋短期與長期動機。

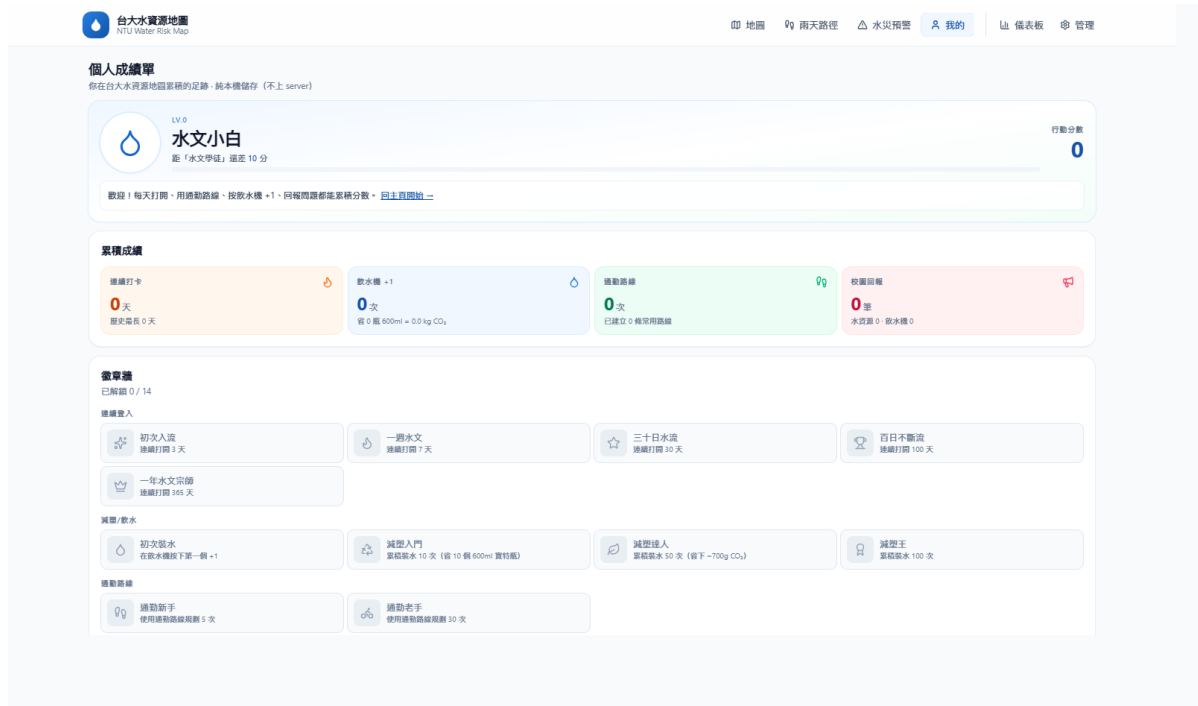


Figure 6 個人成績單畫面：段位（水文學徒 → 水文大師）、4 格累積成績（連續打卡、飲水機 +1、通勤路線、校園回報）、14 枚徽章牆（4 大類別）。所有資料純本機儲存（LocalStorage），零追蹤。

行動分數設計

- **連續打卡 (Streak)**：每天首次打開地圖即 +1；中斷重置。激勵「每天看一眼」。
- **飲水機 +1**：每按一次「我用它裝水了」，行動分數 +2，並累計減塑寶特瓶 / CO₂e。
- **通勤路線使用**：使用 /route 規劃並實際走過 +1。
- **校園回報**：每筆水資源 / 飲水機回報 +5；管理員確認後再 +5。

5. 演算法細節

5.1 校園路網建構：從 OSM 到 Dijkstra-Ready Graph

最大的工程挑戰是**真實校園路網**從哪裡來。校內的 22 個地標可以手寫，但「校門口走到綜合體育館」要經過哪些 OSM highway=footway 才不會穿牆？解法是用 scripts/build-dense-graph.mjs 透過 Overpass API 抓取台大校園 bounding box 內所有 highway tagged ways。

雙層 GeoJSON 設計

- src/data/campus.geojson：22 個**有名稱的地標**（校門口、傳鐘、共教、活大 ...）——給 UI start/end 下拉選單用。
- src/data/campus-paths.geojson：6,133 個路徑節點、7,248 條邊的**密集路網**（ ≈ 3 MB），僅在 server-side 載入。

邊權標註規則（從 OSM tags 推論）

- indoor=yes / tunnel=yes / covered=yes / highway=corridor $\rightarrow$ covered ≈ 0.95
- arcade=yes $\rightarrow$ covered ≈ 0.85 （騎樓）
- 手動 override 區：椰林大道兩側、行政大樓迴廊、活大周邊
- 易積水區（lowLying）：小椰林道、醉月湖、總圖前廣場、舟山路低處

5.2 加權 Dijkstra 與動態回報整合

Dijkstra 實作：使用二元堆 priority queue 的 $O((V + E) \log V)$ 標準版本，但在 cost function 中即時讀取「目前 active 的 flooding / standing_water 回報」並施加懲罰。**亦即：使用者剛在地圖回報的積水，下一個雨天規劃路徑的同學就會自動繞開。**

效益：在中度雨勢（ $r = 0.6$ ）下，防雨建議路徑相較最短路徑可達遮蔽率提升 $\sim 30\%$ ，但只多走 ~ 50 m。 r 為 0 時兩者重疊（不下雨就不繞）。

5.3 Risk Score 與儀表板聚合

風險分組與分數計算邏輯：

$$\text{risk_score} = \text{report_count} \times 2 + \text{high_severity_count} \times 3 + \text{recent_7d_count} \times 2$$

分數 ≥ 9 標記為**高風險**、 ≥ 4 為**中風險**、其餘為低風險。這套規則直覺、無需 ML、可解釋——校方看數據時可立即理解「為什麼這個地點上榜」。

6. 資料來源整合

本平台整合三項外部公開資料與兩項內部資料：

資料源	類型	用途與處理
OpenStreetMap	地理路網 (Overpass)	抓取台大校內所有 <code>highway=footway/path/service</code> ，產生 6,133 節點路網；用 <code>indoor/tunnel/arcade</code> 推論遮蔽率。
中央氣象署 Open-Data	即時氣象	觀測 O-A0003-001 + 預報 F-D0047-061，抓取大安區雨量、降雨機率、氣溫、濕度。無 CWA_API_KEY 時 fallback 至 mock。
北水處 OpenData	飲水機水質	與 OSM <code>amenity=drinking_water</code> 點位融合 (<code>source: osm / official / merged</code>)，含水質採樣時間與官方狀態。
內建 Mock Store	In-memory	demo 模式預設啟用，無 Supabase 也可完整跑通；冷啟動會 reset。
LocalStorage (client)	個人成績	連續打卡、行動分數、徽章狀態純本機，零追蹤。

Table 1 資料源整合一覽。所有外部 API 都由 server-side 代理 (`/api/weather`、`/api/water-stations`)，API key 不暴露在 client。

6.1 部署與工程實踐

工程亮點

- **10 個 API Routes 全 Node runtime**：明確排除 GitHub Pages 等純靜態託管的可能性；選 Vercel hnd1 取得最低台日 RTT。
- **重資料 Server-Only**：3 MB 校園路網檔絕不送 client；使用 `dynamic({ssr:false})` 處理 Leaflet 的 window 依賴。
- `maxDuration:30s`：`/api/route` 與 `/api/forecast` 設置長 timeout 容納 Dijkstra 與 90 天熱力預測。
- **Mock / Real 雙模式**：環境變數一鍵切換，期末展示時零配置上線。
- **Error Boundaries**：頁面級 `error.tsx` + 全域 `global-error.tsx` 雙層保護，避免地圖載入失敗整站白屏。

6.2 API 端點清單

Endpoint	Method	用途
<code>/api/reports</code>	GET / POST	取得 / 新增水資源回報 (含圖片上傳)
<code>/api/reports/[id]</code>	PATCH / DELETE	管理端修改狀態 / 加管理員 note / 刪除
<code>/api/reports/[id]/confirm</code>	POST	眾包確認 (<code>still_exists</code> / <code>resolved</code>)
<code>/api/stats</code>	GET	儀表板聚合 (KPI + 趨勢 + 排行 + 飲水機 SLA)
<code>/api/weather</code>	GET	CWA OpenData 代理，金鑰 server-side 保護
<code>/api/route</code>	POST	雨天 Dijkstra；起終點 + mode + avoidWater
<code>/api/forecast</code>	GET	1 / 3 / 6 小時積水機率格網
<code>/api/water-stations</code>	GET	飲水機點位 (OSM + 北水處融合)
<code>/api/water-stations/[id]/report</code>	POST	飲水機回報 (<code>broken</code> / <code>fixed</code> / <code>refill</code>)
<code>/api/path-geometry</code>	POST	路徑幾何輔助 (<code>snap to road</code>)

7. 視覺設計系統

整套 UI 採用「Apple 等級的克制 + 顏色語意化」原則：

- **品牌色** #0EA5E9 (Sky 500)：呼應水的清澈，主 CTA、活躍連結、品牌徽。
- **嚴重度色階**：● 紅 = 嚴重、● 橙 = 中等、● 藍 = 輕微——與地圖 marker、儀表板 chart、表格 badge 共用，全站視覺語意一致。
- **icon library**：全站 lucide-react 線性圖示，零 emoji，避免跨平台 fallback 差異。
- **字級系統**：標題 11–16 px、內文 12.5–13 px、KPI 數字 30+ px tabular-nums 對齊。
- **動效**：animate-fade-in + ease-soft-out 自訂 curve，整站轉場 250 ms，乾淨不浮誇。

8. 結論與未來工作

本專題完成了一套可上線、可擴充、有實證價值的校園水資源 web app，不只是課堂作業 demo，而是已部署於 Vercel、有 6 個頁面、10 個 API、整合 3 個外部資料源的產品級實作。我們把「水資源管理」這個傳統工程議題，重新詮釋為「資料 + 介面 + 行為」的三層問題，並用最務實的技術組合——Next.js + Leaflet + OSM + Dijkstra ——做出可立即使用的工具。

8.1 未來工作

1. **接入 NTU SSO 或 Magic Link**：取代目前的密碼登入，提升管理端安全性。
2. **推播通知**：當使用者通勤路線上出現新積水回報，主動推送 Web Push。
3. **歷史趨勢回放**：把 90 天熱力預測加上 timeline scrubber，可回看 1 個月前的某天校園水情。
4. **ML 路徑預測**：當回報數 > 1,000 時，可嘗試以 LightGBM 取代手動係數，預測「未來 1 小時哪段路會出現新積水」。
5. **校方 API 對接**：與台大總務處事務組系統 API 串接，已解決的回報自動同步。

參考文獻 References

- [1] Vercel Inc. (2024). *Next.js 14 — The React Framework for the Web*.
<https://nextjs.org/docs>
- [2] Agafonkin, V. (2024). *Leaflet — An Open-Source JavaScript Library for Mobile-Friendly Interactive Maps*.
<https://leafletjs.com/>
- [3] OpenStreetMap Foundation. (2024). *OpenStreetMap Wiki — Map Features and Overpass API Documentation*.
<https://wiki.openstreetmap.org/>
- [4] 交通部中央氣象署 (2024). 氣象資料開放平臺——觀測 O-A0003-001 與 預報 F-D0047-061.
<https://opendata.cwa.gov.tw/>
- [5] 臺北自來水事業處 (2024). 飲水機水質開放資料 *OpenData*.
<https://data.water.gov.taipei/>
- [6] Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1), 269–271.
- [7] Recharts Group. (2024). *Recharts — A composable charting library built on React components*.
<https://recharts.org/>
- [8] Supabase Inc. (2024). *Supabase — The Open Source Firebase Alternative*.
<https://supabase.com/docs>
- [9] Tailwind Labs Inc. (2024). *Tailwind CSS — Utility-first CSS framework*.
<https://tailwindcss.com/>
- [10] Vercel Inc. (2024). *Vercel Platform — Deploy Next.js Apps*.
<https://vercel.com/docs>
- [11] Lucide Contributors. (2024). *Lucide — Beautiful & consistent icon toolkit made by the community*.
<https://lucide.dev/>